

Models

To just hold information. It does zero actual work.

Spring Boot uses your Model as a **molding template** to translate data between three totally different languages: JSON (the internet), Java (your server), and SQL (your database).

- **Internet to Java:** Let's say a mobile app sends a POST request with JSON text: `{"labName": "Robotics"}`. Spring Boot catches this text. It looks at your `Lab` model, uses your **empty constructor** to build a blank Lab box in memory, and then uses your `setLabName("Robotics")` to shove the internet text into the Java box.
- **Java to Database:** Spring Boot then hands that filled box to the Repository. The Repository looks at the `@Table` and `@Column` sticky notes on your Model to know exactly how to rip the data out of the box and write the `INSERT INTO` SQL string

1. What do **models** technically do?

In Java, you can't just pass around raw database rows. Java only understands "Objects."

Technically, a Model (like your `Lab.java` file) is just a custom data type you created. It is a temporary container holding variables (`labId`, `labName`, `building`) in your computer's RAM.

- When data comes **out** of MySQL, Spring Boot stuffs it into this container so your Java code can read it.
- When data comes **in** from the internet, Spring Boot stuffs it into this container so it can hand it to the database.

It doesn't "do" anything. It just holds data in a shape that Java understands.

2. Why did we need all those Annotations again?

Java and MySQL do not speak the same language.

- Java uses `Classes` and `Strings`.
- MySQL uses `Tables` and `VARCHARs`.

The annotations (`@Entity`, `@Table`, `@Id`, `@Column`) are translation instructions for Spring Boot. When Spring Boot starts up, it looks at your `Lab` class. Without the annotations, it just sees a random Java file and ignores it. But when it sees `@Entity`, it says: *"Ah! I need to physically go into MySQL and build a table for this class."* When it sees `@Id`, it says: *"I will make this variable the Primary Key."*

The annotations prevent you from having to write 50 lines of `CREATE TABLE` and `INSERT INTO` SQL code.

Repositories

To talk to the database.

JpaRepository

When Spring Boot's scanner sees `extends JpaRepository`, it says: *"Ah, they want to talk to the `labs` table, but they were too lazy to write the SQL."* So, in the fraction of a second while your server is booting up, **Spring Boot literally writes the code for you in the background**. It generates a hidden, invisible Java class that contains all the raw JDBC `Connection`, `PreparedStatement`, and `ResultSet` code you wrote on day one. It wires that hidden code directly into your server. When your Controller yells `labRepository.findAll()`, it is actually triggering the invisible SQL code that Spring Boot ghost-wrote for you.

Dynamic Proxies

To understand how Spring Boot generates this hidden code, you first have to understand what an `interface` is in standard Java.

An `interface` is just a **contract**. It is a piece of paper that says, *"Whoever signs this contract promises they know how to `save()`, `delete()`, and `findAll()`."* But the contract itself doesn't actually do the work.

In normal Java, *you* (the programmer) have to write a Class that signs the contract and physically writes the code for those methods.

Here is what Spring Boot does instead: When you boot up your server, Spring Boot reads your `LabRepository` interface. It sees the contract. Instead of making *you* write the implementation, Spring Boot uses a high-level Java feature called a **Dynamic Proxy**.

Think of a Dynamic Proxy as an automated ghost-writer. In the fraction of a second during startup, Spring Boot literally writes a brand new, invisible Java class in your RAM. This invisible class "signs" your contract and automatically pastes in all the raw JDBC code (the `PreparedStatement`, `ResultSet`, etc.) needed to talk to MySQL.

Then, Spring Boot secretly swaps out your empty interface with this fully-functioning invisible class. So when you call `labRepository.findAll()`, you are actually running the ghost-writer's code.

If the Model is just a dumb container holding data, the Repository is the exact opposite. It is pure action.

Technically speaking, a Repository is a **Data Access Object (DAO)**. Its entire purpose in software engineering is to isolate all your database code into one specific place so the rest of your app doesn't have to look at raw SQL.

Here is exactly how Spring Boot's repositories work under the hood.

1. What is it physically in your code?

If you look at `LabRepository.java`, you will notice it is not a `class`. It is an `interface`. And it is completely empty. You just wrote: `public interface LabRepository extends JpaRepository<Lab, Integer>`

In standard Java, an empty interface does absolutely nothing. It is just a blank contract. You can't execute an interface.

2. The Technical Magic (Dynamic Proxying)

This is where Spring Boot does its heaviest lifting.

When you start your server, a tool inside Spring Boot (called **Hibernate**) scans your code. It finds your empty `LabRepository` interface and realizes, *"Hey, they want to talk to the `labs` table, but they didn't write any SQL."*

So, in a fraction of a millisecond, Spring Boot generates a hidden, invisible Java class in your computer's RAM. It physically writes all the boilerplate database code for you in the background.

3. How it translates your commands

Because Spring Boot wrote that hidden class, you inherit dozens of pre-built methods. It takes simple Java commands and translates them directly into the raw JDBC code you were writing by hand the other day.

- **When you type:** `labRepository.save(newLab)`
 - **Technically, Spring Boot executes:** `INSERT INTO labs (lab_name, building) VALUES (?, ?)`
 - It opens the connection, grabs the strings from your `Lab` model, replaces the question marks, executes the update, and extracts the new auto-incremented ID.
- **When you type:** `labRepository.findAll()`
 - **Technically, Spring Boot executes:** `SELECT * FROM labs`
 - It runs the query, loops through the `ResultSet`, builds a brand new `Lab` object for every single row it finds, packs them all into a Java `List`, and hands the list to you.
- **When you type:** `labRepository.findById(7)`
 - **Technically, Spring Boot executes:** `SELECT * FROM labs WHERE lab_id = 7`

Technically, a Repository is an **auto-generating code factory**.

You just tell it *which* Model to manage (`Lab`) and what data type the Primary Key is (`Integer`). Spring Boot handles the rest. It completely eliminates the need for you to manually manage database connections, write SQL strings, or handle database errors ever again.

Controllers

These are needed to use CRUD operations and to talk to the outside world.

Technically speaking, your Controller is your **Web Layer**. It is also literally the "C" in the most famous software architecture pattern in the world: **MVC (Model-View-Controller)**.

When you build a Controller, you are creating an **API Endpoint**.

1. **API (Application Programming Interface):** A way for two computers to talk to each other.
2. **Endpoint:** The exact URL address where that conversation happens (e.g., <http://localhost:8080/api/labs>).

Its exact job: It is the HTTP Traffic Cop. The Controller is the *only* part of your application that understands the language of the internet (HTTP requests, Status Codes like "200 OK" or "404 Not Found", and JSON).

When a POST request comes in, the Controller's job is to:

1. Strip away all the internet networking garbage.
2. Translate the JSON payload into a Java POJO (your Model).
3. Hand that POJO to the DAO (your Repository).
4. Take the saved result, package it back up into JSON, and shoot it back across the internet.

In a perfectly designed application, the Controller never speaks SQL, the Repository never speaks JSON, and the Model just happily gets carried back and forth between them.

@RestController: This tells Spring Boot, *"This class is the Front Desk, and it ONLY speaks JSON."* If you used a normal **@Controller**, Spring Boot would try to return a full HTML webpage. **Rest** means "Representational State Transfer," which is just a fancy way of saying "I send and receive raw data (JSON), not websites."

@RequestMapping("/api/workstations"): This sets the base street address for the entire class. It tells Spring Boot, *"If anyone on the internet types <http://localhost:8080/api/workstations>, send them to this specific Front Desk."*

@GetMapping: This tells a specific method to listen for **"Read"** requests. When you type a URL into Google Chrome and hit enter, your browser sends a GET request. It just means *"Give me the data, I don't want to change anything."*

@PostMapping: This tells a specific method to listen for **"Create"** requests. You cannot send a POST request just by typing a URL into a browser. A POST request carries a hidden package of data (like submitting a login form, or using that [api-tests.http](#) file we made).

@RequestBody: This is the magic translator. When a POST request comes in, it carries JSON text. Java doesn't understand JSON. **@RequestBody** catches that JSON text and instantly builds a Java **Workstation** object out of it so your code can use it.

Why does **save()** return the object?

You looked at this code:

```
public Workstation createWorkstation(@RequestBody Workstation newWorkstation){  
    return workstationRepository.save(newWorkstation);  
}
```

And you asked: *"After all it's just saving so what is being returned?"*

This is an incredibly sharp observation. In a lot of older programming languages, saving to a database is a **void** method (it returns nothing). But in Spring Boot, **it returns a Receipt**.

Here is exactly what happens in milliseconds:

1. The Missing ID: When you send a POST request to create a new Workstation, you send something like this: `{"stationName": "New PC", "status": "Available"}` Notice what is missing? **The ID**. You don't know the ID yet because MySQL hasn't auto-incremented it yet.

2. The Save & Update: When Spring Boot's "Ghost Writer" runs `workstationRepository.save(newWorkstation)`, it does two things:

- It runs `INSERT INTO workstations (station_name, status)...`
- It instantly turns around and asks MySQL: *"Hey, what ID did you just give that? Oh, ID number 6?"*
- Spring Boot then slaps `stationId = 6` onto your Java `newWorkstation` object.

3. The Receipt: It then **returns** that updated object back to the Controller, which converts it to JSON and sends it back to the screen.

Why is this useful? Imagine you are using the Instagram app. You post a new photo. The app sends the photo to the server (POST). If the server just saved it and returned **void** (nothing), your phone wouldn't know the ID of the photo you just posted. If you immediately clicked "Delete Photo", the app wouldn't know which photo to tell the database to delete!

By returning the saved object (complete with the new ID generated by MySQL), the frontend mobile app gets a "Receipt." It now knows exactly what the ID is in case the user wants to edit or delete it immediately.

